Safety Protocol Verification of the Canadarm3

Samantha Papais[1]

School of Computing, Queen's University, Kingston, Canada

Abstract

The Canadarm3 will serve as an autonomous robotic system on the Lunar Gateway, performing maintenance, aiding astronauts, and capturing visiting spacecraft. Since the Gateway will not be crewed at all times, verifying the arm's off-nominal behaviours is vital. This model will use hybrid automated planning in PDDL+ to model the arm's functionality, environment, and various protocols it will follow in unlikely scenarios. Previous methods of falsifying such cyber-physical systems used black-box algorithms; however, this model uses a white-box approach in which the functionalities and procedures are known and clearly defined. The falsification of this system takes the form of producing counterexamples to the protocols that have been outlined for the system by intentionally trying to induce failure in the model. Out of the 12 problem files designed to test various scenarios the Canadarm3 may encounter, 7 of them were able to induce failure in the model, indicating that the safety protocols currently in place need revision. The model created does not include some of the key features of the Canadarm3—such as its numerous joints and its second, smaller arm—leaving many avenues for future research and development.

1. Introduction

Canada's contribution to Gateway—a lunar-orbiting space station—will be various robotic applications such as the Canadarm3, which will perform actions including maintenance on the craft, aiding astronauts with spacewalks, and catching visiting spacecraft [1]. Since Gateway will be much farther from Earth than the International Space Station and will not be continuously crewed, autonomous function is a necessary and vital component of its design. The use of automated planning in space robotics has many advantages, such as improved system reliability and cost efficiency. However, these benefits are only gained if the agent can function reliably in all situations, including those that are statistically rare. The ways in which a robotic application handles these unlikely scenarios are called its off-nominal behaviours. The goal of this model is to identify gaps in the safety protocols outlined for the Canadarm3; if a fail state can be induced, then the safety protocols may require revision.

This model will use hybrid automated planning to verify the off-nominal behaviours of the Canadarm3 by modelling the functionalities of the arm, the various environmental factors that may affect it, and the protocols it should follow when encountering an unlikely scenario. The first step will be to model the dynamic environment. This step involves modelling the agent's behaviour and the actions it can take when the environment meets specific criteria. It will also describe how these actions affect the environment in turn. The next procedure will be to model the off-nominal behaviours and test the off-nominal behaviours by using the model to find scenarios from which it cannot recover. These failure states can be found by running simulations designed to induce failures. If a plan for a failure state is found, it may indicate that there is an error in the model or that the off-nominal behaviours are incomplete.

2. Background

There are various domains from which the task of verifying the correctness of safety-critical cyber-physical systems (CPS) stems.

Model checking is one such domain. Model checking [2] is an automatic method for verifying a system model against its formal specification with the goal of finding errors during the process of system design and development. Model checking is often used to find counterexamples in a model's design; that is, to find failure states. Current research in using automated planning to verify CPS measures their results against model checkers.

A connection has been established between model checking and automated planning in previous work, such as in Cimatti et al. (1999), who proposed a new approach to planning using methods from software and hardware model checking [3]; however, the process of using PDDL and other planning languages to falsify CPS has been largely unexplored. Edelkamp (2003) analyzed what PDDL might be able to accomplish as a model-checking tool, as well as possible limitations it might have [4]. Through his experiments, Edelkamp discovered that using planning as a model-checker could introduce a number of benefits, such as higher performance and more concise domain specifications. S. Bogomolov et al. (2014) further connected the use of PDDL and model checking by translating hybrid PDDL+ domains to hybrid automata, allowing them to use hybrid model-checkers to verify their planning problems [5]. Aineto et al. (2022) aimed to explain the behaviour of hybrid systems with

CISC-813: Automated Planning – Final Project

PDDL+ since traditional model-checking tools struggle to produce concrete trajectories [6]. They translated their problem into PDDL+ and used planners to find those trajectories instead. Their results indicated there are advantages, such as greater performance, of using PDDL+ over model-checking tools on complex hybrid systems.

A recent work of Aineto et al. (2023) demonstrated that using PDDL+ to falsify CPS was an effective strategy [7]. One of the biggest limitations of using PDDL as an effective falsification method comes from the fact that falsification algorithms treat CPS as black-box systems due to their complexity. Their method involved using PDDL+ to model a white-box approach. This model most closely builds off of this last approach—it will involve modelling the hybrid domain in PDDL+ and then attempting to create plans that reach a failure state.

3. A Planning Model for Verifying the Safety Protocols of the Canadarm3

This model covers the Canadarm3's ability to catch and dock visiting spacecraft as well as the safety procedures it should follow in case of imminent collisions with crafts and debris. It also models how these collision system protocols interact with the arm's energy system and sensors. PDDL+ was used to model this domain due to the hybrid nature of the problem setting—it requires both discrete and continuous processes that classical planning cannot effectively express.

This model treats physical space as if it is two-dimensional (2D), mostly because ENHSP [8], the planner being used to find counterexamples, is not equipped to efficiently handle three-dimensional (3D) space. The 2D grid is useful for modelling the locations of objects in each scenario. It also makes the use of velocities simple, since they can be represented in the form (x', y'), where x' is the increase of the object's x coordinate in one time instance and y' is increase of the object's y coordinate in one time instance.

There are various discrete actions, continuous processes, and events that work together to simulate the Canadarm3 and its safety protocols. Below is an in-depth description of the various systems of the model, as well as how they work together.

3.1. Docking System

The main functionality of the Canadarm3 presented in this model surrounds docking spacecraft at the various ports of the Lunar Gateway. Ideally (with no complications), the process of catching and docking a craft should look like this [9]:

The Canadarm3 should...

1. detect that there is a craft within sensor range.
2. track the craft.
3. let the craft approach until it is close enough that the arm can reach out and grasp it.
4. let the craft match velocity with the Lunar Gateway (and the arm) so that reaching towards it is a stable and safe operation.
5. extend itself out towards the craft.
6. grasp the craft with its "hand".
7. bring the craft to a free port located on the Lunar Gateway.
8. dock the craft to that port.

This system was modelled with various actions, events and processes, described below in detail.

The actions involved in this system are:

- `track_craft` will "lock on" to a specific craft that it wants to match velocity with, catch, and dock, or avoid altogether.
- `catch_craft` will begin the process of extending the arm to catch the craft.
- `grasp` will use the "hand" on the end of the arm to grasp the spacecraft.
- `dock_craft` will begin the process of bringing the craft to one of the ports on the Lunar Gateway and docking it there.
- `go_to_port` will begin the process of moving the arm and the craft that it is grasping to an empty port.

The processes involved in this system are:

- `approach_station` simulates the craft moving close enough to the Lunar Gateway for the arm to catch it.
- `match_velocity` simulates a visiting spacecraft matching velocity with the Lunar Gateway, allowing the arm to make a safe and stable catch.
- `reach_towards_craft` simulates the continuous action of extending the arm to catch the spacecraft, once the spacecraft has been tracked, has moved within range, and has matched its velocity with the Lunar Gateway.
- `move_to_dock` simulates moving the craft to a free port once the arm has grasped the craft in its hand.
- `moving_to_port` simulates the arm moving itself and the grasped spacecraft to a free port.

The events involved in this system are:

- `object_detected` will trigger for a given object when it has entered the range of the arm's sensor.

- `object_undetected` will trigger for a given, currently detected object when it has exited the range of the arm's sensor.
- `object_untracked` will trigger for a given, currently tracked object when it has exited the range of the arm's sensor.
- `spacecraft_approached` will trigger when the spacecraft has moved within reaching distance of the arm. It indicates that the craft is about to match velocity with the arm.
- `spacecraft_aligned` will trigger when the spacecraft has matched the velocity of the arm. This event indicates the end of the `match_velocity` process and the resulting alignment allows the arm to reach out to grasp the craft.
- `reached_craft` will trigger when the arm has extended to be close enough to the craft to grasp it. This event ends the `reach_towards_craft` process.
- `arrived_at_port` will trigger when the process of moving the arm towards a port to dock a craft is complete. This event indicates that the craft is ready to be docked.

Combined, these actions, events, and processes allow the model to represent catching and docking a spacecraft.

3.2. Collision System

The next system in this model concerns the collision safety protocols of the Canadarm3. The collision system describes how the arm should react when a possible collision is detected with an incoming craft or piece of space debris.

The previous section describes what the protocol of catching and docking a spacecraft should be under normal, predictable conditions. Consider, though, if the incoming craft is approaching too fast and is in danger of colliding with the arm, or if there is space debris floating around that may hit the arm. In these cases, the arm should be able to detect that a collision is imminent and take steps to avoid it. In the safety protocol outlined for this model, the arm's behaviour when there is an imminent collision with a **craft** should be the following:

The Canadarm3 should...

1. detect that a collision with a craft is imminent.
2. trigger its safety mode where all normal functionality is disabled, i.e. if it was in the middle of doing something, it should stop and pay attention to the collision.
3. recognize that it is a collision with a craft that is imminent, not a piece of space debris.
4. notify the craft that a collision is imminent, and the craft should be responsible for stopping itself.
5. exit its safety mode and continue what it was doing before.

If there is an imminent collision with a **piece of debris**, then the protocol is the following:

The Canadarm3 should...

1. detect that a collision with a piece of debris is imminent.
2. trigger its safety mode where all normal functionality is disabled, i.e. if it was in the middle of doing something, it should stop and pay attention to the collision.
3. recognize that it is a collision with a piece of debris that is imminent, not a craft.
4. move 90 degrees to the trajectory of the incoming piece of debris to move out of its way.
5. exit its safety mode and continue what it was doing before.

These safety protocols were modelled with various events and processes, described below in detail.

The various processes involved in this system are:

- `collision_avoidance_debris` will move the arm 90 degrees to the velocity of an incoming piece of debris that it is in danger of colliding with.
- `collision_avoidance_craft` will notify the craft that it must come to a stop to avoid colliding with the arm. It uses a deceleration value to determine how fast it can stop.

The various events involved in this system are:

- `collision_warning` will trigger when a collision between the arm and another object is imminent. It will trigger the safety mode of the arm, preventing it from completing other actions and processes until the collision is safely avoided.
- `additional_collision_warning` is identical to `collision_warning`, except that it does not trigger the safety mode. This event is used when the arm is already in safety mode but another collision is imminent.
- `exit_safety_mode` will exit safety mode once the collision with an object has been avoided. This event indicates that the arm is ready to continue normal execution of actions and processes.
- `collision` is triggered when a collision occurs between the arm and an object. This is considered true when the distance between the arm and the object is less than or equal to 0.01. This event also sets the `(failure-collision ?obj)` predicate to true.

3.3. Energy & Orbit System

The energy of the Canadarm3 is presumed to come from the Lunar Gateway itself, which will likely get energy via solar panels and the light of the Sun. This implies that, in order to model how the Canadarm3 gets energy, we must also model the orbit of the Lunar Gateway around the Moon. When the Lunar Gateway is on the dark side of the Moon, it will not be getting charged—it will rely on its reserve energy. When it is in the Sun, it will be getting charged.

To model the orbit, a countdown system was used. This involved one process and two events:

- `orbit_countdown` is a process that is always active. It counts down by 1 every time instance starting from some some initial value.
- `toggle_sun_on` is triggered when the countdown has reached 0 and the Lunar Gateway was previously in shadow. It also resets the counter to its original starting value.
- `toggle_sun_off` is triggered when the countdown has reached 0 and the Lunar Gateway was previously in sunlight. It also resets the counter to its original starting value.

The battery system is affected by whether or not the Lunar Gateway is in sunlight or not. It also has a mechanism for when it is on low battery (less than or equal to 20% of the maximum battery capacity), where it disables non-vital functions of the arm—only the collision avoidance system is still active in this state. When the battery is low, it drains at half of the normal rate.

There are various processes involved in modelling the battery system:

- `battery_draining_normal` models the normal drain rate of the battery when the Lunar Gateway is in shadow.
- `battery_draining_critical` models the drain rate of the battery when the Lunar Gateway is in shadow and also has less than 20% of its total battery life.
- `battery_charging` models the charge rate of the battery when the Lunar Gateway is in the sunlight.

There are various events involved in modelling the battery system:

- `battery_low` triggers when the battery level falls below 20% of its total battery life.
- `battery_sufficient` triggers when the battery level rises above 20% of its total battery life.
- `battery_drained` triggers when the battery level falls to 0. This also triggers the `(failure-battery-drained)` predicate to be true.

3.4. Sensor System

The sensor system is the smallest of the various systems described. In this model, when a collision occurs, the sensor on the arm breaks. The sensor is presumed to be responsible for detecting objects in the arm's surroundings and using this information to carry out tasks and avoid collisions. If the sensor is broken, these tasks become impossible. Here, it is modelled that if a sensor breaks, the arm must simply wait for a human to fix it. Until then, it cannot do anything.

This system involves one process and two events:

- `sensor_repair_countdown` is a process that counts down by 1 every time instance starting from some some initial value. It is triggered by the `(sensor-functional)` predicate being switched to false.
- `sensor_broken` is an event that triggers when a collision occurs. It sets the `(sensor-functional)` predicate to false.
- `sensor_fixed` is an event that triggers after the countdown has reached 0. It models a human completing the repair of the sensor, allowing the arm to resume normal functionality. It also resets the counter to its initial value, in case another collision occurs and the arm must be repaired again.

3.5. Extra Events and Processes

There are several processes and events that do not fit into any of the above systems, but are still vital for the functioning of the model. They concern the movement of objects in space:

- `move_x` is a process that moves everything that has a velocity in the x direction.
- `move_y` is a process that moves everything that has a velocity in the y direction.
- `stopped_moving` is an event that sets an object's velocity to 0 if it is moving sufficiently slowly (below 0.0025 in both the x and y direction).

3.6. Failure States

There are 2 main failure states in this model that will attempt to be reached when falsifying the system:

1. `(failure-collision ?obj)` is the predicate made true when the arm has collided with either a craft or a piece of debris. This will, in turn, set `(sensor-functional)` to be false. Multiple collisions can be accumulated by using the `(num-collisions)` function in the goal state to set how many collisions should occur before the situation is considered a failure.

2. `(failure-battery-drained)` is the predicate triggered when the arm has run out of battery. This failure could lead to collisions, creating a domino effect of failures.

4. Evaluation

To falsify the system, various problem files were created. The ENHSP-2020 [8] planner was used in the `planutils` environment [10] to create plans for these problem files.

The following details the various problem files used to falsify the domain and identify gaps in the previously described safety protocols. Problems 1-5 test the collision system with crafts. Problems 6-8 test the collision system with debris. Problem 9 tests the collision systems with a mix of debris and spacecraft. Problem 10 tests the battery system on its own, while Problem 11 tests the battery system when paired with an incoming spacecraft. Problem 12 tests the sensor system with an incoming spacecraft and a piece of debris.

In every problem, these values remain the same:

1. The arm's starting position is (30, 30) and its initial velocity is (0, 0).
2. There is a port where spacecraft can dock at (40, 30).
3. The range of the arm's sensor is 20.

4.1. Problem 1

Important Objects	Important Start State Predicates & Function Values	Goal State
craft1 starting position = (0, 0) velocity = (1, 1) deceleration rate = -0.25	collision detection distance = 3	(successful-dock craft1)

Problem 1 acted as a test file to ensure that the model could handle successfully docking a craft at a port; thus, it does not count as a falsification task since its goal was not failure. It was instead compared with Problem 2, which used the same problem file, except that its goal was to find a possible failure while docking the craft. The problem involves a single spacecraft moving linearly towards the arm with a velocity of (1, 1). The planner was able to find a plan to successfully dock the spacecraft.

4.2. Problem 2

Important Objects	Important Start State Predicates & Function Values	Goal State
craft1 starting position = (0, 0) velocity = (1, 1) deceleration rate = -0.25	collision detection distance = 3	(failure-collision craft1)

Using the same starting state as with Problem 1, Problem 2's goal state was to induce a collision failure. After letting the planner run for 600 seconds (10 minutes), it timed out, indicating that a plan to induce failure in the model was not easily discoverable. Since the problem is unsolvable (at least, in the time frame given), it can be tentatively said that failure cannot be induced for these initial values and set of objects.

4.3. Problem 3

Important Objects	Important Start State Predicates & Function Values	Goal State
craft1 starting position = (0, 0) velocity = (5, 5) deceleration rate = -0.01	collision detection distance = 3	(failure-collision craft1)

Problem 3 features a craft moving at a faster velocity than the previous problems. Additionally, the craft deceleration rate was changed from -0.25 to -0.01. The planner was able to induce failure in this scenario. One of the reasons why failure occurs here is because the collision detection distance is less than the velocity per time step. This naturally means that the Lunar Gateway does not have time to react and tell the incoming spacecraft to slow down. To remedy this, the collision detection distance should be set sufficiently high to detect spacecraft arriving at a reasonable velocity.

4.4. Problem 4

Important Objects	Important Start State Predicates & Function Values	Goal State
craft1 starting position = (0, 0) velocity = (5, 5) deceleration rate = -0.01	collision detection distance = 7	(failure-collision craft1)

Continuing from the previous problem, in Problem 4, the collision detection distance was raised from 3 to 7, a significant increase. Despite this change, the planner is still able to induce failure. The reason for the collision in this instance is the inability of the quick-travelling spacecraft to slow down in time before hitting the arm; therefore, even though the collision detection distance is sufficient, the craft is unable to slow down in time to avoid a collision. Essentially, the deceleration rate is too low for the high speed of the craft. The plan generated for Problem 4 looks similar to the plan generated for Problem 3, except the imminent collision is detected earlier, reflecting the change in the collision detection distance.

4.5. Problem 5

Important Objects	Important Start State Predicates & Function Values	Goal State
craft1 starting position = (0, 0) velocity = (5, 5) deceleration rate = -0.25	collision detection distance = 7	(failure-collision craft1)

In Problem 5, the deceleration rate has been changed to have a more immediate effect, meaning the craft will come to a stop faster. The deceleration rate has been changed from -0.01 to -0.25; however, this change is still not enough for the craft to be stopped in time. In fact, it would take a far more significant increase for the craft to

avoid hitting the arm—it would take an almost instantaneous stop to avoid the collision at this velocity. A craft coming in at a reasonable speed (in this model, under a velocity of (3, 3)) should be able to stop in time with these values.

However, the purpose of safety validation is to account for edge cases. Given that the current model cannot handle crafts arriving at a high velocity—a situation that is entirely possible—the safety protocols must be altered to account for this scenario. For example, the safety protocol could be changed for the incoming craft so that there is some threshold distance from where it arrives at the arm where it must decrease its speed to some reasonable value. Additionally, the Canadarm3's sensor range could be increased, as well as its collision detection distance. Another possible solution could be to have the spacecraft stop almost instantly when a collision with the arm is detected. This could potentially be done by having the craft fire its thrusters in the opposite direction of its travel to stop as quickly as possible, though the force exerted would need to be greater than or equal to the force propelling it into the arm. These methods could even be combined for a more airtight safety protocol.

4.6. Problem 6

Important Objects	Important Start State Predicates & Function Values	Goal State
debris1 starting position = (0, 0) velocity = (2, 2)	collision detection distance = 7	(failure-collision debris1)

Problem 6 begins the testing of debris collisions. The Canadarm3 is now responsible for moving out of the way to avoid a collision, since the debris cannot control itself like a spacecraft can. For this problem, the spacecraft object was replaced with a debris object, moving at a reasonable speed towards the arm at a velocity of (2, 2). The protocol for an imminent debris collision is for the Canadarm3 to move 90 degrees to the velocity of the debris. This problem was found to be unsolvable by the planner, indicating that the Canadarm3 was able to move out of the way and no collision occurred.

4.7. Problem 7

Important Objects	Important Start State Predicates & Function Values	Goal State
debris1 starting position = (0, 0) velocity = (2, 2) debris2 starting position = (0, 30) velocity = (1, 0)	collision detection distance = 7	(or (failure-collision debris1) (failure-collision debris2))

Problem 7 contains 2 pieces of debris, coming in from different trajectories, at different speeds. One piece comes in from (0, 0) at a velocity of (2, 2), and the other piece of debris comes in from (0, 30) at a velocity of (1, 0). The planner says that this problem is also unsolvable, indicating that the Canadarm3 is able to dodge both of these pieces of debris without a collision occurring.

4.8. Problem 8

Important Objects	Important Start State Predicates & Function Values	Goal State
debris1 starting position = (0, 30) velocity = (2, 0) debris2 starting position = (30, 0) velocity = (0, 2) debris3 starting position (30, 60) velocity = (0, -2)	collision detection distance = 7	(or (failure-collision debris1) (failure-collision debris2) (failure-collision debris3))

In Problem 8, we explore a scenario where the arm will not be able to dodge 3 incoming pieces of debris. Consider a piece of debris coming in from (30, 0) with a velocity of (0, 2), a second piece coming in from (0, 30) with a velocity of (2, 0), and a third coming in from (30, 60) with a velocity of (0, -2). The safety protocol for avoiding debris fails in this scenario, because, while the arm is able to move out of the way of one of the pieces of debris, it is not able to guarantee that it is out of the way of the other pieces in time to avoid a collision. This is especially true when all pieces of debris are expected to arrive at the same time (as in this problem).

For this problem, it is interesting to consider how the Canadarm3 could move if this model were in 3D. It would also have the option to move in the up or down directions to avoid debris, allowing for more options in avoiding collisions; however, debris could also arrive from these directions.

4.9. Problem 9

Important Objects	Important Start State Predicates & Function Values	Goal State
debris1 starting position = (0, 30) velocity = (2, 0) craft1 starting position = (0, 0) velocity = (2, 2) deceleration rate = -0.25	collision detection distance = 7	(failure-collision debris1)

Problem 9 combines the craft and debris collision avoidance protocols in a simple way. In this problem, there is one spacecraft and one piece of debris, both travelling at a low speed but set to arrive at the Canadarm3 at the same time. The planner ran for 600 seconds before timing out, unable to find a path to failure. This is likely because the craft is responsible for stopping itself when a collision is imminent, leaving the Canadarm3 able to successfully dodge the incoming debris.

4.10. Problem 10

Important Objects	Important Start State Predicates & Function Values	Goal State
	battery level = 10 battery drain rate = 3 battery charge rate = 1 orbit clock = 5 orbit clock counter = 5	(failure-battery-drained)

There is not much that can be done about the orbit of the station around the Moon and the amount of time the arm is able to be charged by the Sun's light. Thus, the safety testing of ensuring that the arm does not run out of energy mainly concerns the rate at which the arm loses charge versus the rate at which it charges. Basically, failure is only possible in this regard if the arm drains battery when it is in shadow faster than it charges energy when it is in the light.

A limitation of this model is that it does not account for the amount of energy that each action of the arm takes to execute. This would be an interesting extension that could add complexity to the planner's ability to falsify it. For now, we assume that every action takes the same amount of energy to execute as every other action.

In Problem 10, we demonstrate how failure can eventually be achieved if the battery drain rate is higher than the battery charge rate. The drain rate has been set to 3 while the charge rate has been set to 1 with an initial battery level of 10. If left long enough, the arm will eventually run out of power. If out of power, the Canadarm3 cannot avoid collisions, leaving it open to damage and further failure states. The planner was able to create a plan that reflects this exact scenario.

4.11. Problem 11

Important Objects	Important Start State Predicates & Function Values	Goal State
debris1 starting position = (0, 0) velocity = (0.5, 0.5)	collision detection distance = 7 battery level = 10 battery drain rate = 3 battery charge rate = 1 orbit clock = 5 orbit clock counter = 5	(and (failure-battery-drained) (failure-collision debris1))

Problem 11 reflects the other kind of failure that can occur when the arm runs out of power: a collision occurs because the arm does not have enough power to move itself out of the way of the incoming object. The goal state for this problem is that the battery must be drained and a collision failure occurs. In this problem, we use similar values to Problem 6 where we had a piece of debris starting at (0, 0) moving with velocity (2, 2). In Problem 6, no collision occurred; however, if the Canadarm3 has run out of energy, the arm will not be able to move out of the way. The planner was able to induce failure in the model for this problem.

4.12. Problem 12

Important Objects	Important Start State Predicates & Function Values	Goal State
debris1 starting position = (20, 20) velocity = (1, 1) craft1 starting position = (25, 25) velocity = (5, 5) deceleration rate = -0.25	collision detection distance = 7	(and (= (num-collisions) 2) (not (sensor-functional)))

The final system in the model is the sensor system. When a collision occurs, the arm's sensor breaks. The repair timer indicates the amount of time it takes for a human to fix it. While the sensor is not functional, the Canadarm3 cannot execute its normal functions (aside from charging), leaving it vulnerable to debris collisions.

The purpose of testing this sensor system as its designed is mostly to indicate how fast humans must be to fix the sensor when it breaks to avoid collisions with debris of different incoming speeds. In Problem 12, the sensor breaks from an initial collision with a quickly arriving spacecraft. Though the following piece of debris is moving slow enough for the Canadarm3 to theoretically dodge it, due to the sensor being offline, a collision occurs. Thus, the planner was able to find a plan to two separate collision failures due to the sensor being broken by the first collision.

5. Related Work

There is little recent research specifically exploring the use of PDDL+ for falsifying CPS. The most relevant contribution in this area is by Aineto et al. (2023), who presented one of the first attempts to apply automated planning with PDDL+ for system falsification [7]. As discussed previously, they used PDDL+ to model hybrid systems and find counterexamples that violate the systems' safety requirements. They were able to demonstrate that PDDL+-based falsification performs competitively against Falstar, a state-of-the-art falsification tool.

Beyond using PDDL+, other recent research has employed various related methods to verify CPS. For example, Zhang et al. (2018) proposed a two-layered falsification framework guided by Monte Carlo Tree Search (MCTS) [11]. This method efficiently explores continuous input spaces to identify behaviors that lead to property violations; however, it still considers the hybrid system to be a black-box, similar to previous methods of model checking. MCTS balances the exploration and exploitation of the input space effectively, demonstrated in their results by the out-performance of MCTS over other search strategies.

Akazaki et al. (2018) introduced a deep reinforcement learning approach to falsification, turning the problem into a learning task in which an agent discovers adversarial inputs that trigger unsafe system states [12]. Like the

approach taken by this model, they used their agent to find counterexamples to the intended functionality of the CPS. Due to the nature of reinforcement learning, this approach also treats the CPS like a black-box, where the agent learning about specific inputs and actions would lead to a violation. One of their main goals was to reduce the number of simulations while also deeply exploring the problem space. Like most other black-box methods, this approach is limited in its ability to be easily interpreted and understood.

6. Summary

This work is motivated by the need to ensure the safe and reliable autonomous operation of Canadarm3. As the system is intended to function with minimal human intervention, it is critical to verify that its control logic and safety protocols are robust against potential failure scenarios. Previous work has demonstrated that using PDDL+ to falsify CPS is both a viable and efficient strategy. Building on this foundation, the goal of this project is to falsify the Canadarm3 by modelling it in PDDL+ and attempting to identify plans that would lead the system to a failure state. To achieve this goal, the functionality and safety protocols of Canadarm3 were modeled in PDDL+. The ENHSP planner was then used to search for plans that could violate the specified safety constraints, thereby revealing potential failure states within the modeled system.

The problem files created for this model were designed to test the functionality of the Canadarm3. For each problem file, this resulted in one of two outcomes: the planner was able to find a plan to failure, or it was not. When the planner was unable to find a plan for a given scenario, this indicated that the safety protocols implemented are appropriate and sufficient for the functioning of the arm in that specific scenario. Though, it is worth noting that this result does not necessarily guarantee the robustness of the Canadarm3's off-nominal behaviours in those scenarios. With a timeout set to 600 seconds, the planner may not have had enough time to find a plan because the problem space was too large. In general, it is very difficult to demonstrate complete safety with large, complex domains.

The other case was that the planner was able to find a plan—that is, the planner was able to induce failure in the system. If the planner was able to find a plan, then that means that the safety protocols implemented in the system are not sufficient, and will have to be revised. This was the case in 7 out of the 12 problem files created, meaning that there is plenty of room for improvement in the design of the Canadarm3's off-nominal behaviours. This also indicates that the model is doing what it is intended to do—find gaps in the off-nominal behaviours of the Canadarm3. It is worthwhile to note that this relies on the assumption that the model is implemented correctly and with safety protocols that are somewhat reasonable for the system.

6.1. Future Work

This model does not encompass the full capability of the Canadarm3, nor all of the safety protocols it would require to be considered safe. As such, there are several limitations that could serve as motivation for future work.

One major limitation is that the model does not take into account the second, smaller arm that will be attached to the Canadarm3. This smaller arm is intended to repair the larger arm and reduce the need for astronaut spacewalks, but it is not represented in the current model.

Another limitation is that the model only considers motion in a 2D space. In reality, the Canadarm3 will operate in 3D, providing additional flexibility and options for maneuvering. Related to this, the model treats the arm as if it does not have joints, aside from a single swivel point where it is attached to the Lunar Gateway. In practice, the Canadarm3 will have seven degrees of freedom, allowing for much more complex motion [1].

The collision avoidance assumptions in the model are also highly simplified. Objects are assumed to have no size when performing collision avoidance, which is unrealistic. Additionally, collision detection is very limited: the model can only detect a future collision if the velocities of two objects are pointed at each other instead of away, and if the objects are within a certain distance of each other. In future work, this should be improved by calculating whether there exists a point in time at which the trajectories of two objects intersect, regardless of their instantaneous velocity directions. Though, it is worthwhile to mention that this task is non-trivial in PDDL+. Moreover, collision avoidance treats the arm as if it is not attached to the station, effectively considering only the hand and ignoring the rest of the arm and its attachment point when checking for potential collisions.

Finally, the generated plans were unable to be validated due to domain parsing issues. It appears that the validator could not handle the exponent symbol (^) properly. While ENHSP was able to parse the domain without issue, running the validate option in the VAL tool [13] in both VS Code and `planutils` failed on any line containing an exponent symbol. Since much of the domain relies on distance calculations between objects to ensure they are moving in the correct directions, this issue made validation impossible.

References

[1] C. S. Agency, Canadarm, canadarm2, and canadarm3 – a comparative table, 2024. URL: https://www.asc-csa.gc.ca/eng/iss/canadarm2/canadarm-canadarm2-canadarm3-comparative-table.asp.

[2] E. M. Clarke, Model checking, in: International conference on foundations of software technology and theoretical computer science, Springer, 1997, pp. 54–56.

[3] A. Cimatti, E. Giunchiglia, F. Giunchiglia, P. Traverso, Planning via model checking: A decision procedure for ar, 1999. doi:10.1007/3-540-63912-8_81.

[4] S. Edelkamp, Limits and possibilities of pddl for model checking software, in: International Conference on Automated Planning and Scheduling, 2003.

[5] S. Bogomolov, D. Magazzeni, A. Podelski, M. Wehrle, Planning as model checking in hybrid domains, Proceedings of the AAAI Conference on Artificial Intelligence 28 (2014). URL: https://ojs.aaai.org/index.php/AAAI/article/view/9037. doi:10.1609/aaai.v28i1.9037.

[6] D. Aineto, E. Onaindia, M. Ramirez, E. Scala, I. Serina, Explaining the behaviour of hybrid systems with pddl+ planning, in: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22), 2022, pp. 4542–4548. doi:10.24963/ijcai.2022/631.

[7] D. Aineto, E. Scala, E. Onaindia, I. Serina, Falsification of cyber-physical systems using pddl+ planning, Proceedings of the International Conference on Automated Planning and Scheduling 33 (2023) 2–6. URL: https://ojs.aaai.org/index.php/ICAPS/article/view/27172. doi:10.1609/icaps.v33i1.27172.

[8] E. Scala, P. Haslum, S. Thiébaux, M. Ramirez, Interval-based relaxation for general numeric planning (2016).

[9] C. S. Agency, Canadarm2's cosmic catches, 2025. URL: https://www.asc-csa.gc.ca/eng/iss/canadarm2/cosmic-catches.asp.

[10] C. Muise, F. Pommerening, J. Seipp, M. Katz, Planutils: Bringing planning to the masses, in: ICAPS 2022 System Demonstrations, 2022.

[11] Z. Zhang, G. Ernst, S. Sedwards, P. Arcaini, I. Hasuo, Two-layered falsification of hybrid systems guided by monte carlo tree search, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 37 (2018) 2894–2905.

[12] T. Akazaki, S. Liu, Y. Yamagata, Y. Duan, J. Hao, Falsification of cyber-physical systems using deep reinforcement learning, in: International Symposium on Formal Methods, Springer, 2018, pp. 456–465.

[13] R. Howey, D. Long, M. Fox, Val: automatic plan validation, continuous effects and mixed initiative planning using pddl, in: 16th IEEE International Conference on Tools with Artificial Intelligence, 2004, pp. 294–301. doi:10.1109/ICTAI.2004.120.